

reception area may have their own group address. Then, these can be controlled by sending a message to that group address.

One reason why group address is used even when the group has only one device in it is that the messaging system is based around the concept of publish and subscribe. To illustrate the way this works, consider lights and light switches. For example, I install some light switches in some part of the building and I want to control them from a particular switch or group of switches. I would have each of those lights subscribed to a particular address. On the other hand, I will tell the switch to publish a message to that particular address. When I flick the light switch, it will send a message to the destination address and all the lights subscribed to that address will respond. These will all switch off or on, get dim or brighter, or something like that.

The beauty of this technique is that the light switch does not know anything about the light it is sending a message to. And the light itself does not know anything about the light switch that is controlling it. So, I can make changes by adding more lights to the building and subscribe to that same address. Or, I can remove them, and it will hardly have any impact on configuration. I do not need to reconfigure the various devices because these do not know enough to be affected by that change.

Bringing together the above concepts gives us models, which are specifications for software components. The collection of models determines the things a device can do.

A model will specify what messages it supports, what state values are associated with those messages and what transition changes in those state values should be accomplished. So, a model brings everything together, and a typical product will support multiple



Bluetooth connectivity

models.

Now, current specification for version 1 has a number of separate lists of models. The first list is called generics—generic models are those that are applicable to any type of product having a simple behaviour such as on and off. There is a big list of such generic models, which can be controlled in a mesh network, wirelessly.

Beyond the generics list, the current version has a list of models specifically for lighting because it is really important, in addition to sensor networks comprising any type of sensor.

There are also things to do with automation and time because scheduling events in a mesh network is very important.

Bluetooth Mesh for smart buildings

Automation is important in smart buildings and smart homes.

Bluetooth Mesh defines the scene as a mechanism by which you can build automation. Imagine walking into a room. You will most probably want a lot of things to happen in response to that event, like lights to come on or maybe get brighter, air-conditioner to liven up or shutters to come down. The

point here is that there is a whole collection of changes spanning different devices and systems that should all take place in a single orchestrated manner in response to that one occupancy event indicated by the sensor. The Bluetooth Mesh mechanism lets you do this.

Switch, light and sensor models have a series of Bluetooth Mesh standard models defined by the specifications inside them. Lights can be switched on or off, so the generic model server will go inside them. Light switches control lights by switching them on or off, so there will be the generic on/off client model inside them.

The difference between a client and a server is that the server has state values, whereas the client does not. Client just sends and receives messages. It can change the state of the server, and the device having the server model will respond by switching on or off.

Light lightness server, or LLS, controls brightness and dimming of lights, while light hue saturation lightness server, or LHSL, is for colour control. Light lighting control server, or LLCS, allows interfacing of sensors with light(s) to accomplish useful effects.

State bindings are relationships between state items. If a particular state changes its value through state binding, then a change can automatically be triggered in another state inside the device in another model, perhaps, for which this special relationship has been specified.

Now, imagine the occupancy sensor detecting people entering and leaving a room. When you walk into the room, the sensor is sampling occupancy data and will send a message using the sensor model to LLCS to indicate occupancy. LLCS gets to know about this occupancy by registering its value in the state server. Thanks to the state binding server